

Ayudantía 6: análisis de redes y ruteo geoespacial

NetworkX, OSMnx, coordenadas, rutas

IIC2115

Objetivo

La idea de hoy

entender desde datos espaciales hasta una ruta o una métrica de red interpretable.

- ▶ Entender qué representa un grafo en problemas geoespaciales.
- ▶ Distinguir qué hace NetworkX y qué aporta OSMnx.
- ▶ Ordenar mentalmente el flujo: coordenadas → nodos → ruta → visualización.
- ▶ Evitar errores típicos: latitud/longitud, nodos iguales, dirección de calles, pesos y CRS.

Qué fue lo que más apareció en el ticket

Dificultades más frecuentes

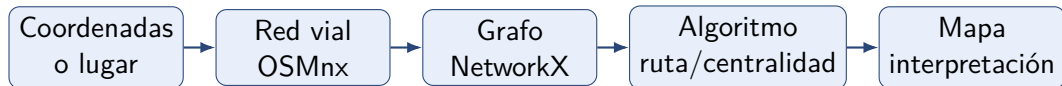
- ▶ Coordenadas, rutas, path y nodos cercanos.
- ▶ Entender el flujo del código y qué hace cada función.
- ▶ Uso de OSMnx, dependencias y descarga de datos.
- ▶ Visualización de mapas y rutas.

Entonces:

La traba principal es conectar varias capas:

**geometría + grafos + librerías +
visualización**

Mapa general del flujo



Les puede servir para ordenarse, pero obviamente no es una guía estricta para todos los ejercicios

Primero: ¿qué es un grafo?

Componentes

- ▶ **Nodos:** intersecciones, estaciones, comunas, puntos.
- ▶ **Arcos:** conexiones entre nodos, calles, relaciones.
- ▶ **Atributos:** longitud, nombre de calle, velocidad, tipo de vía.

En una red vial

- ▶ Nodo $\approx$ cruce o punto de la calle.
- ▶ Arco $\approx$ segmento transitable.
- ▶ Peso $\approx$ costo de moverse: distancia, tiempo, dificultad.

El grafo no es el mapa: es una estructura para calcular sobre el mapa.

NetworkX: crear una red pequeña

```
import networkx as nx

G = nx.Graph()           # grafo no dirigido
G.add_node("A")          # nodo individual
G.add_nodes_from(["B", "C"])

G.add_edge("A", "B")      # arco A-B
G.add_edge("B", "C")      # arco B-C
```

Lectura metodológica

- ▶ G es el objeto que almacena la red.
- ▶ add_node agrega entidades.
- ▶ add_edge define relaciones entre entidades.
- ▶ Luego se pueden aplicar algoritmos sobre G.

Grafo dirigido vs no dirigido

No dirigido: `nx.Graph()`

- ▶ Si puedo ir de A a B, también puedo ir de B a A.
- ▶ Útil para caminatas o redes simplificadas.

Dirigido: `nx.DiGraph()`

- ▶ $A \rightarrow B$ no implica $B \rightarrow A$.
- ▶ Útil para calles con sentido, flujos, restricciones.

Error frecuente

Si una ruta existe en un sentido pero no en el otro, puede ser por la **dirección** del grafo. En red vial de autos, eso puede ser correcto.

Del tablero al grafo: la idea del primer notebook

Tablero

```
X 0 X 0 0
0 0 X 0 X
X 0 0 0 0
```

Interpretación

- ▶ Cada 0 puede ser un nodo transitable.
- ▶ Cada X es un obstáculo.
- ▶ Dos nodos se conectan si son vecinos válidos.

La lógica es la misma que en una ciudad: puntos transitables conectados por segmentos transitables.

Algoritmos de conectividad y resiliencia

Preguntas que responde NetworkX

- ▶ ¿La red está conectada completa?
- ▶ ¿Hay componentes separadas?
- ▶ ¿Qué nodos son críticos?
- ▶ ¿Qué arcos, si se cortan, desconectan la red?

```
nx.is_connected(G)  
nx.connected_components(G)  
nx.articulation_points(G)  
nx.bridges(G)
```

En transporte, esto permite hablar de **puntos críticos** y **robustez** de la red.

Centralidad: tres formas de decir “importante”

Métrica	Pregunta	Interpretación urbana
Grado	¿Cuántos vecinos tiene?	Cruces con muchas conexiones
Cercanía	¿Qué tan cerca está del resto?	Buena accesibilidad global
Intermediación	¿Cuántas rutas pasan por él?	Posible cuello de botella

Ojo

Una centralidad alta no siempre significa “mejor”. Puede significar más uso, más dependencia o más vulnerabilidad.

OSMnx: ¿qué aporta?

Idea clave

OSMnx descarga datos de OpenStreetMap y los transforma en un grafo compatible con NetworkX.

```
import osmnx as ox

G = ox.graph_from_place(
    "Macul, Santiago, Chile",
    network_type="drive"
)
```

- ▶ NetworkX: estructura y algoritmos de grafos.
- ▶ OSMnx: descarga, simplifica, proyecta y visualiza redes reales.

Formas de descargar una red

Método	Cuándo usarlo	Cuidado principal
Lugar	Comuna, barrio o ciudad	nombre ambiguo
Punto + radio	zona alrededor de una coordenada	orden lat/lon
Bounding box	rectángulo definido manualmente	orden de límites
Polígono	área exacta	geometría válida

Mensaje importante

No hay una única forma “correcta”. La mejor depende de cuánto control necesitan sobre el área de estudio.

Coordenadas: el enredo más común

Latitud

- ▶ Norte-sur.
- ▶ En Santiago suele ser negativa.
- ▶ Ejemplo: $-33,486$.

Longitud

- ▶ Este-oeste.
- ▶ En Chile suele ser negativa.
- ▶ Ejemplo: $-70,599$.

Regla para no perderse

Muchas funciones reciben (lat, lon) para puntos humanos, pero `nearest_nodes` pide **X = longitud** e **Y = latitud**.

De coordenadas a nodos del grafo

```
# Punto de origen y destino en formato humano: latitud, longitud
origen = (-33.486, -70.599)
destino = (-33.500, -70.620)

# nearest_nodes usa X=longitud, Y=latitud
orig_node = ox.distance.nearest_nodes(G, X=origen[1], Y=origen[0])
dest_node = ox.distance.nearest_nodes(G, X=destino[1], Y=destino[0])
```

Si los nodos salen iguales

- ▶ Los puntos pueden estar demasiado cerca.
- ▶ Se pudo invertir latitud y longitud.
- ▶ El grafo descargado puede no cubrir esos puntos.

Ruta mínima: la receta mental

```
ruta = nx.shortest_path(  
    G,  
    source=orig_node,  
    target=dest_node,  
    weight="length"  
)
```

Qué significa cada parte

- ▶ source: nodo inicial.
- ▶ target: nodo final.
- ▶ weight="length": minimizar distancia, no cantidad de arcos.

Sin weight, el camino más corto puede ser el de menos segmentos, no necesariamente el de menor distancia.

Visualizar una ruta

```
fig, ax = ox.plot_graph_route(  
    G,  
    ruta,  
    route_color="red",  
    route_linewidth=4,  
    node_size=0  
)
```

Metodología para depurar

Antes de graficar, imprimir:

- ▶ cantidad de nodos de la ruta,
- ▶ nodo inicial y final,
- ▶ largo total aproximado,
- ▶ si el origen y destino están dentro del grafo.

Calcular largo de una ruta

```
lengths = ox.routing.route_to_gdf(G, ruta)["length"]
largo_total = lengths.sum()
print(largo_total)
```

Interpretación

La ruta es una lista de nodos. Para calcular distancia, hay que mirar los **arcos** que conectan esos nodos y sumar sus atributos de longitud.

La pregunta no es solo “cuál es la ruta”, sino **cuánto cuesta recorrerla**.

Geometría vs grafo: no son lo mismo

Concepto	GeoPandas	NetworkX / OSMnx
Objeto básico	geometrías	nodos y arcos
Pregunta típica	¿dónde está?	¿cómo se conecta?
Operaciones	buffers, joins, áreas	caminos, centralidad, conectividad
Salida común	mapa / capa espacial	ruta / métrica de red

Puente entre ambos mundos

OSMnx permite convertir partes del grafo a GeoDataFrames para visualizar, exportar o cruzar con datos espaciales.

CRS y proyección: cuándo preocuparse

Idea simple

Las coordenadas latitud/longitud sirven para ubicar puntos en la Tierra, pero no siempre son cómodas para medir distancias o áreas directamente.

- ▶ Para ruteo con OSMnx, muchas redes ya traen atributo `length` en metros.
- ▶ Para medir geometrías con GeoPandas, conviene proyectar a un CRS en metros.
- ▶ Si el mapa se ve raro o las distancias no tienen sentido, revisar CRS.

Pregunta de control: ¿mis unidades son grados o metros?

Ejemplos de uso: grafos aleatorios

Para qué sirven en el notebook 2

Los grafos aleatorios permiten probar algoritmos sin depender de una ciudad real.

- ▶ Se generan grafos conexos de distinto tamaño.
- ▶ Se visualizan varias estructuras juntas.
- ▶ Se prueba cómo crece el costo computacional.

Antes de llevar un algoritmo a Macul, probarlo en un grafo chico.

TSP: por qué la fuerza bruta explota

Problema del viajero

Buscar una ruta que visite todos los nodos minimizando costo.

Número de órdenes posibles $\approx n!$

Nodos	Rutas candidatas aproximadas
4	24
7	5.040
10	3.628.800

Mensaje

Aunque el código sea correcto, puede ser inviable para redes grandes.

Checklist para enfrentar un ejercicio de redes

1. ¿Qué representa cada nodo?
2. ¿Qué representa cada arco?
3. ¿El grafo debe ser dirigido o no dirigido?
4. ¿Cuál es el peso/costo relevante?
5. ¿Mis coordenadas están en el orden correcto?
6. ¿El área descargada contiene origen y destino?
7. ¿Estoy minimizando distancia, tiempo u otra cosa?
8. ¿Cómo interpreto el resultado en el contexto del problema?

Errores frecuentes y cómo diagnosticarlos

Síntoma	Posible causa	Qué revisar
Nodos iguales	puntos cercanos o lat/lon invertido	imprimir coordenadas y nodos
No hay ruta	grafo dirigido o zona desconectada	probar no dirigido / revisar cobertura
Mapa vacío	bbox incorrecto	orden de límites
Ruta extraña	peso ausente o equivocado	usar <code>weight="length"</code>
Distancia rara	CRS/unidades	grados vs metros
Error al descargar	dependencia/red/nombre ambiguo	instalar, cambiar método

Caso

Quiero una ruta caminable entre dos puntos de una comuna. ¿Qué decisiones debo tomar antes de calcular la ruta?

- ▶ Área de descarga: lugar, punto+radio, bbox o polígono.
- ▶ Tipo de red: walk, drive u otro filtro. CLAVE buscar custom filter y saber cómo usarlo. Puede ayudar muchísimo.
- ▶ Origen y destino: coordenadas correctamente ordenadas.
- ▶ Peso: distancia, tiempo u otra variable disponible.
- ▶ Visualización: grafo completo, ruta o ambas.